

RISC-V User Mode + Machine Mode 前置知识说明

1. 为什么要实现 User Mode 和 Machine Mode?

在最简单的 RV32I CPU 中，所有程序都可以看成运行在一个“裸机最高权限”环境中。这个阶段只需要处理：

普通整数指令
load/store
branch/jump
ecall/mret 的简化形式
基本 CSR

但是一旦要更接近真实 RISC-V 特权架构，就需要区分不同的特权级。

RISC-V 常见特权级有：

特权级	编码	名称	典型用途
U-mode	00	User mode	普通用户程序
S-mode	01	Supervisor mode	操作系统内核
M-mode	11	Machine mode	最高权限，裸机固件、启动代码、异常处理

本实验只实现：

U-mode + M-mode

不实现 S-mode。

也就是说，CPU 中只存在两种运行状态：

```
current_priv = U  
current_priv = M
```

其中：

M-mode 权限最高，可以访问机器级 CSR，可以配置 trap 入口，可以通过 mret 切换到 U-mode。
U-mode 权限最低，只能执行普通指令，不能访问 M-mode CSR，不能执行 mret。

2. User Mode 和 Machine Mode 的直观区别

2.1 M-mode 能做什么？

M-mode 是最高权限，可以：

1. 执行普通 RV32I 指令。
2. 读写 M-mode CSR，例如 `mstatus`、`mtvec`、`mepc`、`mcause`。
3. 执行 `mret`。
4. 配置异常入口 `mtvec`。
5. 设置 `mepc`，然后通过 `mret` 跳转到指定程序。
6. 处理来自 U-mode 的 `ecall`、非法指令、访存异常等 `trap`。
7. 配置 PMP 或其他内存保护机制。

M-mode 可以理解成：

CPU 的“管理者模式”。

2.2 U-mode 能做什么？

U-mode 可以执行普通程序，例如：

```
add
sub
lw
sw
beq
jal
jalr
fence
fence.i
ecall
ebreak
```

但是 U-mode 不能执行或访问：

```
mret
csrw mstatus, ...
csrw mtvec, ...
csrw mepc, ...
csrw mcause, ...
pmpcfg0 / pmpaddr0 等机器级 CSR
```

U-mode 访问 M-mode CSR 或执行 M-mode 特权指令时，应该触发：

```
Illegal instruction exception  
mcause = 2
```

U-mode 可以理解成：

被 M-mode 管理的普通应用程序模式。

3. Trap 是什么？

`trap` 是异常和中断的统称。

本实验主要处理异常，不重点处理外部中断。

常见异常包括：

异常	mcause
Instruction address misaligned	0
Instruction access fault	1
Illegal instruction	2
Breakpoint	3
Load address misaligned	4
Load access fault	5
Store address misaligned	6
Store access fault	7
Environment call from U-mode	8
Environment call from M-mode	11

当发生 trap 时，CPU 需要：

1. 保存异常指令的 PC 到 mepc。
2. 保存异常原因到 mcause。
3. 保存异常附加信息到 mtval。
4. 修改 mstatus，记录 trap 前的特权级和中断使能状态。
5. 当前特权级切换到 M-mode。

- 6. PC 跳转到 mtvec。
- 7. flush 异常指令之后的年轻指令。

4. 精确异常是什么？

五级流水线中，某一拍可能同时存在多条指令：

IF 取指
ID 译码
EX 执行
MEM 访存
WB 写回

如果 EX 阶段的指令触发异常，那么 IF 和 ID 阶段可能已经取到了后面的指令。

精确异常要求：

1. 异常指令之前的老指令正常完成。
2. 异常指令之后的年轻指令全部取消。
3. 异常指令本身不能产生错误副作用。
4. mepc 精确保存异常指令的 PC。

例如：

```
add x1, x2, x3
ecall
addi x5, x5, 1
```

执行 `ecall` 后：

add 应该完成。
ecall 进入 trap。
addi 不应该写 x5。

这就是为什么 trap 必须配合流水线 flush 和副作用取消。

5. CSR 是什么？

CSR 是：

Control and Status Register 控制与状态寄存器

它们不是普通寄存器 `x0 ~ x31`，而是 CPU 内部控制状态。

本实验中最重要 CSR 有：

CSR	地址	作用
<code>mstatus</code>	<code>0x300</code>	保存全局状态、MPP、MIE、MPIE
<code>misa</code>	<code>0x301</code>	声明 CPU 支持的 ISA 和特权能力
<code>medeleg</code>	<code>0x302</code>	异常委托，本实验可固定为 0
<code>mideleg</code>	<code>0x303</code>	中断委托，本实验可固定为 0
<code>mie</code>	<code>0x304</code>	机器模式中断使能
<code>mtvec</code>	<code>0x305</code>	M-mode trap 入口地址
<code>mcounteren</code>	<code>0x306</code>	控制 U-mode 是否可读计数器
<code>mscratch</code>	<code>0x340</code>	M-mode trap handler 临时寄存器
<code>mepc</code>	<code>0x341</code>	trap 返回地址
<code>mcause</code>	<code>0x342</code>	trap 原因
<code>mtval</code>	<code>0x343</code>	trap 附加信息
<code>mip</code>	<code>0x344</code>	机器模式中断 pending
<code>pmpcfg0</code>	<code>0x3A0</code>	PMP 配置
<code>pmpaddr0</code>	<code>0x3B0</code>	PMP 地址
<code>mcycle</code>	<code>0xB00</code>	机器模式 cycle 计数器低 32 位
<code>minstret</code>	<code>0xB02</code>	机器模式退休指令计数器低 32 位
<code>mcycleh</code>	<code>0xB80</code>	机器模式 cycle 计数器高 32 位
<code>minstreth</code>	<code>0xB82</code>	机器模式退休指令计数器高 32 位
<code>cycle</code>	<code>0xC00</code>	用户可读 cycle 计数器
<code>instret</code>	<code>0xC02</code>	用户可读退休指令计数器
<code>cycleh</code>	<code>0xC80</code>	用户可读 cycle 高 32 位
<code>instreth</code>	<code>0xC82</code>	用户可读 instret 高 32 位
<code>mhartid</code>	<code>0xF14</code>	hart 编号，单核通常为 0

6. mstatus 中最重要的位

本实验重点实现 `msstatus` 的以下位：

位	名称	作用
3	<code>MIE</code>	M-mode 全局中断使能
7	<code>MPIE</code>	trap 前的 MIE 保存位
12:11	<code>MPP</code>	trap 前的特权级

本实验没有 S-mode，所以 `MPP` 只支持：

```
00 = U-mode  
11 = M-mode
```

不支持：

```
01 = S-mode
```

当发生 trap 时：

```
mstatus.MPP <= trap 前的 current_priv  
mstatus.MPIE <= mstatus.MIE  
mstatus.MIE <= 0  
current_priv <= M-mode
```

当执行 `mret` 时：

```
current_priv <= mstatus.MPP  
mstatus.MIE <= mstatus.MPIE  
mstatus.MPIE <= 1  
mstatus.MPP <= U-mode  
PC <= mepc
```

注意：实现 U-mode 之后，`mstatus.MPP` 不能再硬连线为 M-mode。它必须允许被写成 U-mode，否则 M-mode 无法通过 `mret` 进入 U-mode。

7. M-mode 如何进入 U-mode?

M-mode 进入 U-mode 的典型流程是：

```

la t0, user_entry
csrw mepc, t0

csrr t1, mstatus
li t2, ~((3 << 11))
and t1, t1, t2  # 清除 MPP, 使 MPP = 00, 即 U-mode
csrw mstatus, t1

mret

```

执行 `mret` 后：

```

PC = user_entry
current_priv = U-mode

```

所以：

```

mepc 决定 mret 跳到哪里。
mstatus.MPP 决定 mret 后进入哪个特权级。

```

8. U-mode 如何回到 M-mode?

U-mode 不能直接执行 `mret` 回 M-mode。

U-mode 通常通过：

```
ecall
```

主动请求 M-mode 服务。

如果当前是 U-mode，执行 `ecall` 后：

```

mepc  = ecall 的 PC
mcause = 8
mtval = 0
PC    = mtvec
current_priv = M-mode

```

如果当前是 M-mode，执行 `ecall` 后：

```
mcause = 11
```

所以 `ecall` 的异常号取决于当前特权级。

9. mret、ecall、ebreak 的区别

指令	当前模式	作用
<code>ecall</code>	U-mode	进入 M-mode trap, <code>mcause = 8</code>
<code>ecall</code>	M-mode	进入 M-mode trap, <code>mcause = 11</code>
<code>ebreak</code>	U/M	断点异常, <code>mcause = 3</code>
<code>mret</code>	M-mode	从 trap 返回到 <code>mepc</code>
<code>mret</code>	U-mode	非法指令, <code>mcause = 2</code>

10. U-mode 下 CSR 访问权限

CSR 地址本身携带权限信息：

`csr[11:10]` 表示读写属性：

11 = read-only

其他 = read/write

`csr[9:8]` 表示最低访问特权级：

00 = U-level CSR

01 = S-level CSR

11 = M-level CSR

访问规则：

如果 `current_priv < csr_min_priv`, 则非法。

如果要写 read-only CSR, 则非法。

如果 CSR 未实现, 则非法。

U-mode 可以访问的典型 CSR：

```
cycle
cycleh
instret
instreth
```

但还要受 `mcounteren` 控制。

例如：

```
mcounteren[0] = 1, U-mode 才能读 cycle/cycleh。  
mcounteren[2] = 1, U-mode 才能读 instret/instreth。
```

如果 `mcounteren` 没开，U-mode 读这些 CSR 也应该触发非法指令异常。

11. I-Cache / D-Cache 对 User Mode 的影响

实现 U-mode 后，Cache 不能只关心中断和缺失，还要关心权限。

11.1 I-Cache 取指权限

U-mode 取指时，需要判断该地址是否允许 U-mode 执行。

如果不允许：

```
mcause = 1  
Instruction access fault
```

并且：

```
不能向 I-Cache 发起正常取指请求。  
不能把非法地址的数据填入 I-Cache。
```

11.2 D-Cache 访存权限

U-mode 执行 load/store 时，需要判断该地址是否允许 U-mode 读写。

如果 load 不允许：

```
mcause = 5  
Load access fault
```

如果 store 不允许：

```
mcause = 7
Store access fault
```

并且：

```
load 不能写 rd。
store 不能写 D-Cache / memory。
```

11.3 权限检查和 Cache hit 的关系

权限检查必须在 Cache 命中返回之前生效。

也就是说，即使 I-Cache 或 D-Cache 已经命中，也不能绕过权限检查。

错误做法：

```
Cache hit 后直接返回数据，不看 current_priv。
```

正确做法：

```
每次 fetch/load/store 都先做权限检查。
权限通过后，才允许访问 Cache。
```

12. fence 和 fence.i 在双模式下的作用

`fence` 和 `fence.i` 都是普通程序可以执行的指令。

fence

`fence` 保证：

```
fence 前的访存完成之后，
fence 后的访存才能继续。
```

有 D-Cache 时，保守实现：

```
暂停流水线。
等待 D-Cache 不 busy。
```

如果是 write-back D-Cache, 则 flush dirty line。
完成后继续执行 fence 后面的指令。

fence.i

fence.i 保证:

前面的数据写入对后续取指可见。

有 I-Cache 和 D-Cache 时, 保守实现:

等待 D-Cache 完成。
write-back D-Cache 写回 dirty line。
invalidate I-Cache。
flush 前端流水线。
从 fence.i 的下一条重新取指。

User mode 下执行 fence.i 也应该允许, 除非你的实验特意要求把它陷入 M-mode 模拟。

13. 本实验的关键目标

最终 CPU 应该做到:

1. M-mode 可以配置 mtvec、mepc、mstatus。
2. M-mode 可以通过 mret 进入 U-mode。
3. U-mode 可以执行普通 RV32I 指令。
4. U-mode 执行 ecall 会 trap 到 M-mode, mcause = 8。
5. M-mode 执行 ecall 会 trap 到 M-mode, mcause = 11。
6. U-mode 访问 M-mode CSR 会触发 illegal instruction。
7. U-mode 执行 mret 会触发 illegal instruction。
8. trap 后 mepc、mcause、mtval 正确。
9. mret 后 current_priv 和 PC 正确恢复。
10. I-Cache / D-Cache 访问不能绕过权限检查。
11. fence / fence.i 能正确同步 Cache。